CONCEPT

One Job

A single package and a single launcher. Drop files in a folder. The job installs what it needs, runs every file, and repairs only from a fixed menu.

1. What it is

One Job is one product, not a stack of custom-node packs. It is a portable job: a folder or zip that describes a finished ComfyUI task, plus a launcher that is the only program a beginner has to open. The workflow graph stays inside the package. The user does not assemble nodes, hunt models, or queue prompts.

Opening the job once prepares the machine. After that, the interface is a folder. Files dropped in are processed. Results appear in an output folder. The ComfyUI canvas is the exception path, used when someone wants to edit the job, not when someone wants to run it.

The name is a working title. The constraint is the point: one entry, one manifest, one repair menu.

2. The problem it replaces

ComfyUI can already install missing nodes, download models, and batch a folder. Each of those is a different extension, a different click path, and a different failure mode. A daily user can stitch them together. A beginner cannot, and should not have to.

Photoshop droplets work because the action and the host are already installed. Dropping a file only means “run this known action.” A ComfyUI graph is a few kilobytes, but it depends on custom nodes, Python packages, and multi-gigabyte checkpoints and LoRAs that may be renamed, gated, or only available from Civitai or Hugging Face. No file extension removes those downloads. A product can hide the clicks and refuse the unsafe ones.

3. What it is not

It is not a new exclusive file type that only ComfyUI understands. OS associations, sharing sites, ComfyUI Manager, and cloud hosts will ignore a private extension. The package is an ordinary folder or zip. The workflow inside it is the API-format JSON ComfyUI already saves. The new part is a manifest beside that JSON, and a launcher that consumes both.

It is not a general agent with a shell. It does not invent pip commands, rewrite arbitrary graphs, or “manage ComfyUI.” It chooses among tools that were shipped with the job.

It does not bundle the models. They are too large, and often licensed. The package references them by filename, folder type, URL, and hash.

4. The package

A job is a directory. Zipping it is optional and changes nothing about how it runs.

File	Role
workflow.api.json	The graph in ComfyUI API format. This is what gets queued.
manifest.json	Nodes by registry id and commit. Models by filename, folder, URL, and hash. Input and output rules. Hardware variants. Repair menu.
input/	The drop folder. Images, and later video or text, that match the job’s extensions.

output/	Finished files, named from the input stem so a rerun can skip work already done.
state/	Local only. Chosen variant, hashes verified, per-file status, repair log. Not required to share the job.

The manifest is the contract. It says what may enter, what must come out, which nodes and models are allowed, how much VRAM and time a file may cost, and which repairs are legal. If a model URL is missing, the job cannot pretend to install it. If a node is not pinned, the launcher will not ask Manager for “whatever matches the name.”

5. The launcher

One program. First open of a job runs preparation. Later opens only watch the folder.

Prepare, in order

Read the manifest. Find or start ComfyUI. Install each pinned node pack at the pinned commit. For each model, search ComfyUI’s real model folders, including paths from extra_model_paths. If the file exists and the hash matches, skip it. If the name matches and the hash does not, stop. If it is missing, download to the correct folder, write to a partial file, verify the hash, then rename. Gated downloads need a token the user already stored. The package never ships someone else’s key. Probe the GPU once, pick a shipped variant, and write that choice into state/.

Run

Watch input/. For each new file of an allowed type, queue the API workflow with that file substituted in. Write the result to output/ under the same stem. Move or mark the source so it is not run twice. If output already exists, skip. One file in memory at a time. A crash leaves finished files intact.

The user can still open the graph. That is an editor action, separate from the drop folder. Saving an edit rewrites the workflow and, if nodes or models changed, the manifest. The drop folder keeps using the last prepared contract until preparation is run again.

6. Errors

Most failures are missing files, hash mismatches, node version conflicts, a full disk, a busy port, an authentication wall, a bad input, or an out-of-memory run. Those are not language problems. They are prevented by the contract, then reported as codes.

Every step returns a code: missing_node, node_version_conflict, missing_model, hash_mismatch, auth_required, disk_full, port_busy, oom, bad_input, output_exists. The launcher branches on the code. A log line is a fallback for what the step could not classify, not the control path.

Deterministic checks never go through a model. Hash, path, “output already exists,” and “file type not allowed” are rules. The repair menu is also a rule: for each code, a list of allowed actions. Typical actions are install this pinned node, download this URL and check this hash, switch to the lower-VRAM variant, skip this file, or ask the user. No action outside that list can run.

7. The local operator

A small on-device model sits behind the rules, and only wakes when a code is missing or a log is messy. Needle 3 is the reference shape: a few tens of megabytes, tool calls with grammar-locked JSON, and a confidence score. The same role can be filled by any model that can only emit a call from a declared list.

Tools are narrow. Read the manifest. Check disk. Install a node by id and version. Download a model to a destination and require a hash. Queue a file. Skip a file. Switch variant. Ask the user. There is no shell.

High confidence and a known code: run the tool. Mid confidence: show the proposed call and wait. Empty or low confidence: stop. A required argument with no evidence is withheld, not guessed.

Fine-tuning is on this product's failures, not on ComfyUI in general. Each failed run stores a structured record: node, model, resolution, VRAM peak, error code, and the action that fixed it. Those rows teach the operator to map a state to one allowed call. They do not teach it to invent a fix. Self-healing means choosing a shipped variant.

8. How far the same idea goes

The job, not the graph, is the thing you share. A finished package carries the contract and, if the author allows it, a repair log with no images attached. The next machine skips models it already has and starts from the variant that worked on similar hardware. Matching "this looks like the 12 GB out-of-memory we already mapped" can stay on device.

Variants are authored, not discovered. Full weight and quantized, batch 1 and batch 4, with and without an optional custom node. The probe picks among those. It does not design a new graph.

Past that, the gains get thin. The operator will not design a better img2img graph, negotiate a license, or debug a custom node's Python. Those stay human, or an explicit cloud step the manifest names. The ceiling is a job you can hand to someone who has never opened ComfyUI: it installs only what the hash says, runs every file in the folder, repairs from a fixed menu, and asks once when it is unsure.

9. What the person sees

First launch: a short prepare screen. Missing models with sizes. A token field if a download is gated. One confirm. A progress line per node and per file. Then the drop folder opens.

Daily use: drop files. A tray icon shows queued, running, done, skipped. Failures show the code in plain language and, if the operator proposed a repair, that one action with yes or no.

Authoring, for the person who already lives in ComfyUI: build the graph as usual, export the job, and the launcher writes the manifest from the graph, then asks for any URL or hash it could not resolve. Publishing a job is that folder. Receiving a job is opening it.

10. Done when

A beginner can open a job, confirm the downloads, drop a folder of images, and collect outputs without opening the node graph. A second machine with the same models does not download them again. A hash mismatch stops instead of silently loading the wrong file. An out-of-memory run can switch to a shipped smaller variant or skip, and cannot run an arbitrary command. The canvas still opens for the author. Nothing else is required to call the product finished.

One package. One launcher. Pinned nodes and hashed models. A drop folder. A fixed repair menu. A small local model only for the cases the rules cannot classify.